SYNCONIC (Pvt) Ltd · Google AI Builder Cup 2026 · Theme: Manufacturing

Shapewise

Architecture, modules and build plan for a manufacturing business system that shapes itself around each factory, changes when you ask, and can be run by people or AI agents.

1 Product and scope
2 Design principles
3 System architecture
4 The layered blueprint
5 Command pipeline
6 Data architecture
7 Approval engine
8 AI architecture
9 MCP design
10 Module catalogue
11 Build order
12 Three-day schedule
13 Evaluation and cut list
14 Cup compliance
15 Risks and open decisions

Version 1.0

Date 15 October 2026

Stack Next.js · Firebase · Cloud Run

AI Gemini · Gemma · ADK · MCP

1 Product and scope

Shapewise is a business system for small and medium manufacturers. The owner describes the factory and uploads what they already use: spreadsheets, photographed job cards, QC sheets and invoices. Shapewise becomes a working system for that factory, covering sales, purchasing, inventory, production and accounting, with their data already inside it.

What it does

Capability	What the owner experiences
Build from description and files	An interview of up to 8 questions plus file uploads produces a full system in about a minute.
Run the factory	Sales orders, purchase orders, goods receipts, production jobs by stage, QC, stock, invoices, payments, and the books.
Change by asking	"Add a dyeing stage", "POs over 500k need my approval". The change is previewed, applied without losing data, and can be undone.
Insights and recommendations	What to produce this week, what to reorder, where scrap and defects come from (by machine, shift, operator).
Delegate to AI agents	Every factory gets its own MCP server. Agents operate within roles, and risky actions wait for approval.

Dynamic for any manufacturer

The engines know manufacturing in general. The blueprint knows the specific industry. The same runtime serves every column below with no industry code.

	Plastics moulder	Garment unit	Food processor	Furniture workshop
Stages	Mixing, moulding, trimming, packing	Cutting, sewing, finishing, packing	Cleaning, cooking, filling, sealing	Cutting, joinery, sanding, polishing
Mode	Batch to stock	Make to order, size grid	Batch, lots, expiry	Job shop
Specs	Resin grade, colour, weight	Style, size, fabric	Recipe, shelf life, allergens	Wood, dimensions, finish
QC	Weight tolerance, flash	Stitch count, measurements	Temperature, pH, seal	Joint strength, finish
Resources	Machines, moulds	Lines, operators	Vats, ovens	Craftsmen, tools
Key metric	Scrap % by machine	Pieces per operator-hour	Yield %, near-expiry	Jobs late vs promised

Prototype scope

In the 3-day prototype

- Onboarding interview, file upload (XLSX, CSV, Sheets, photos)
- Blueprint generation with validation and repair
- Sales: order, delivery, invoice, receipt
- Purchasing: PO, goods receipt, bill, payment
- Inventory with locations, lots optional
- Production: single-level BOM, stages, QC, outputs, scrap
- Accounting: automatic postings, AR, AP, P&L, stock value
- Change by conversation with preview and undo
- Dynamic approvals, audit log
- Assistant, insights agent, per-business MCP

Roadmap (deck only)

- Quotations, partial deliveries, returns
- Multi-level BOM, MRP, capacity planning
- Labour and overhead costing, FIFO valuation
- Bank reconciliation, full tax packs per country
- Voice onboarding with Gemini Live
- Machine maintenance pack, IoT counters
- Mobile shop-floor app, offline mode
- OAuth MCP, per-tool scopes, usage billing
- Industry pack marketplace for partners
- Self-hosted Gemma option for regulated tenants

2 Design principles

Principle	What it means in the build
1. Never generate what must never be wrong	Money, stock, numbering, permissions, approvals and audit live in fixed, tested engines. AI can never change how they count.
2. Configuration, not code	AI produces a validated blueprint (JSON). One runtime executes every blueprint. No generated code anywhere.
3. One writer	Every write from the web app, the assistant or an MCP agent goes through one command pipeline. One set of rules for everyone.
4. Every change is previewed and reversible	Blueprint changes become versioned diffs with a data-impact preview and an undo point.
5. Humans and agents follow the same rules	An agent can never do what its role can't, and risky actions always need approval.
6. Start from reality	Learn from the owner's own files, not only from what they remember to say.
7. Google-native	Firebase and Google Cloud for every service; Gemini and Gemma for every AI task.
8. Designed, not generated, UI	One fixed design system for every blueprint: neutral palette, one accent, two font weights, dense tables, no gradients or decoration.

3 System architecture

3.1 Layers

People and agents	Owner · supervisors · store keeper · accountant · built-in assistant · any MCP client (Gemini ADK agent, Claude, ChatGPT)
Interfaces	Web app (generated screens) · assistant panel · change studio · MCP server (generated tools)
Command pipeline	Authorise → approval policy → engine → computed fields → audit → outbox, in one Firestore transaction
Blueprint (AI-shaped)	Base manufacturing blueprint + industry pack + this factory's customisations: records, fields, stages, workflows, rules, approvals, roles, screens, documents, dashboards
Core engines (fixed)	Items and units · inventory · BOM and production · sales · purchasing · ledger · numbering and documents · permissions · approvals · audit
Google platform	Firestore · Firebase Auth · Cloud Storage · App Hosting · Cloud Run · Cloud Tasks · Cloud Functions · Vertex AI · Secret Manager

3.2 Services

Service	Responsibility	Runs on
web	All screens, server actions calling the runtime, auth session, file upload, realtime listeners	Next.js 16 on Firebase App Hosting (Cloud Run + CDN)
agents	Architect agent (interview, files, blueprint, changes), operator assistant, insights agent	ADK (TypeScript) on Cloud Run
mcp	`/mcp/{tenant}` Streamable HTTP endpoint, token auth, role-filtered tools, rate limits	Cloud Run
worker	Imports, backfills, PDFs, embeddings, outbox relay, nightly integrity checks	Cloud Run (Cloud Tasks targets) + Cloud Functions 2nd gen (Firestore triggers)
gemma	Gemma 4 for bulk column typing and value cleaning during import	Cloud Run with GPU (fallback: Vertex AI Model Garden, then Flash-Lite)

All services share one TypeScript monorepo and one region (e.g. `asia-south1`). Each has its own least-privilege service account.

3.3 Repository

```
apps/
  web/          Next.js 16: landing, onboarding, workspace, change studio, approvals, agents,
  agents/       ADK agents: architect, operator, insights
  mcp/          MCP server
  worker/       jobs and Firestore triggers
packages/
  blueprint/    schema, catalogues, expression language, layers + merge, validator, compiler,
  engines/      items, inventory, production, sales, purchasing, ledger, numbering, documents,
  runtime/      command pipeline, record service, query planner, rollups, outbox, compiled-blu
  tools/        blueprint → tool manifest (JSON Schema) shared by MCP and assistant
  ai/           Gen AI client (Vertex), model router, structured output + repair, prompts, dat
  db/           Firestore paths, converters, tenant transaction helper, Security Rules, index
  ui/           design system, data table, form renderer, kanban, calendar, charts, chat, diff
  blueprints/   base manufacturing blueprint, industry packs, eval fixtures
  config/       tsconfig, eslint, tailwind preset
```

Dependencies point one way: `apps → runtime, tools, ai → engines, blueprint → db`. Engines and blueprint import no framework or vendor SDK, so they are unit tested in isolation.

3.4 Key flows

Onboarding

Sign in → Interview + uploads → Files read (Flash, Gemma) → Business summary confirmed → Pick industry pack → Customisations generated (Pro) → Validate + repair → Owner reviews → Import data → MCP live

Change by conversation

Request (chat or MCP) → Change planner drafts overlay diff → Merge + validate → Migration planner: impact + reversibility → Preview → Owner approves → Backfill job if needed → New version active → Screens + tools update

Agent action through MCP

Token → tenant + role → List allowed tools → Call tool → Command pipeline → Done, or pending_approval → Audit with client name

Production run

Sales order (MTO) or plan (MTS) → Production job from BOM → Issue materials → Stages + QC → Receive outputs, co-products, scrap → Close: variance + cost → Deliver + invoice

4 The layered blueprint

The blueprint is the single description of a factory's shape. It is the only thing AI is allowed to write. Each factory's live blueprint is built from three layers.

1 · Base blueprint	Written and tested by us. Sales, purchasing, inventory, production, accounting, approvals, standard roles, default screens and dashboards.
2 · Industry pack	Plastics, garments, food, furniture... Stages, specs, QC checklists, units, metrics. Picked by AI or generated for a new industry.
3 · Customisations	This factory's own changes, from onboarding and every later request. Stored as a list of overlay operations, not a copy.

4.1 Merge and versioning

- Everything in the blueprint has a **stable ID** (`fld_7k2` , `ent_job` , `stg_dye`). Keys and labels can change; IDs never do.
- Packs and customisations are **overlay operations**: `add` , `update` , `remove` , `disable` , `move` , each targeting an ID.
- `live = merge(base@v, pack@v, overlays)` , then validated and compiled. Every applied change creates an immutable version with author, reason and undo pointer.
- When we improve the base or a pack, factories get the improvement on their next merge. If an overlay touches something the base removed, the change is flagged for review rather than applied silently.

4.2 What can change, and what can't

Dynamic add, edit, remove or turn off	Fixed enforced by engines
Fields on any record, including computed fields	How a posted document affects the ledger and stock
Modules (add "Machine maintenance", turn off "Quotations")	Double entry, gap-free numbering, tax maths
Submodules and steps (skip GRNs, add a dyeing stage)	Posted records are reversed, never edited
Workflows: states, transitions, required fields per step	Accounting can be simplified in the UI, not turned off
Approval flows, conditions, approvers, order of steps	Safety floors on approvals can be tightened, never loosened
Roles, permissions, row-level scopes	Permission checks can't be skipped by any caller
Rules: discounts, validations, alerts, auto-calculations	Audit log is append only
Screens, dashboards, reports, document templates, number formats	Units, quantities and money stored as exact integers

4.3 Blueprint shape (abridged)

```
{
  "id": "bp_dpplastics", "version": 12, "base": "manufacturing@3", "pack": "plastics@2",
  "business": { "name": "DP Plastics", "currency": "LKR", "locale": "en-LK", "timezone": "Asia
  "modules": [ { "id": "mod_sales", "binding": "sales", "enabled": true },
               { "id": "mod_quotes", "binding": "sales.quotation", "enabled": false },
               { "id": "mod_maint", "label": "Machine maintenance", "entities": ["ent_machine"
  "entities": [ { "id": "ent_job", "key": "production_job", "binding": "production.job",
      "fields": [ { "id": "fld_mould", "key": "mould_no", "type": "ref", "to": "ent_mould", "f
                  { "id": "fld_cav", "key": "cavities", "type": "integer", "min": 1 } ],
      "stages": [ { "id": "stg_mix", "label": "Mixing" }, { "id": "stg_mould", "label": "Mould
                    "qc": [ { "id": "qc_w", "label": "Weight", "type": "number", "tolerance":
  "rules": [ { "id": "rl_1", "when": "sales_order.customer.order_count ⩾ 5", "then": { "disco
  "approvals": [ { "id": "ap_po", "on": "purchase_order.submit", "when": "total > 500000",
                   "steps": [ { "role": "production_manager" }, { "role": "owner" } ] } ],
  "roles": [ { "id": "rl_sup", "key": "supervisor", "can": ["production_job.*:own_line"] } ],
  "views": [ { "entity": "ent_job", "type": "kanban", "by": "stage" } ],
  "dashboard": [ "scrap_pct_by_machine", "jobs_late", "stock_below_reorder", "receivables_over
}
```

4.4 Validator rules

- IDs and keys unique; references point to existing entities; only catalogue field types and expression functions.
- Entities touching money or stock must bind to an engine and can't redefine its posting behaviour.
- Workflows reachable and terminating; transitions name existing states; required fields exist.
- Approval and permission rules can't drop below engine safety floors.
- Expressions are parsed and type-checked by our own safe language; no arbitrary code.
- Every error carries a JSON path and a fix hint, which the repair loop feeds back to Gemini.

5 Command pipeline

The single writer for everything. This is what keeps the system consistent no matter how many surfaces or agents are added.

```
Web UI ──────┐
Assistant ───┼──▶ execute(command, actor)
MCP agent ───┘
   1  resolve tenant + actor (uid or token) + channel (web | assistant | mcp)
   2  load compiled blueprint (cached by tenant + version)
   3  validate input against compiled schema
   4  authorise: role permission + row scope
   5  approval policy: matches? → write approval request, return pending_approval
   6  idempotency key check
   7  Firestore transaction:
        read affected docs, check versions
        engine logic (stock, production, ledger, numbering)
        update computed dependents + rollups
        write audit entry + outbox event
   8  commit → return result (or retry on contention)
```

Command kind	Examples
record.create / update	Create customer, update a job's mould number
record.transition	Move a job from moulding to trimming (checks required fields and QC)
engine action	post_invoice, receive_goods, issue_materials, receive_output, record_payment, adjust_stock
approval.decide	Approve or reject; on approve, the original command is re-validated and executed
blueprint.apply	Apply an approved change; creates a version and possibly a backfill job

6 Data architecture

6.1 Firestore layout

```
tenants/{tid}                          profile, activeBlueprintVersion
  members/{uid}                        roleKey, scopes
  blueprints/{version}                 merged + compiled blueprint (immutable)
  overlays/{id}                        customisation operations
  changes/{id}                         request, diff, preview, status, undo pointer
  records/{rid}                        entityKey, data{fieldId: value}, idx{slots}, status, sta
                                       version, links[], searchTokens[], embedding
  items/{id}                           materials, products, units, BOM refs
  boms/{id}                            inputs, outputs, co-products, scrap %, version
  stockLevels/{item_location_lot}      on-hand projection
  stockMoves/{id}                      append only
  journal/{id}                         balanced entry with lines (append only)
  accounts/{id}                        chart of accounts
  sequences/{docKey}                   gap-free counters
  approvals/{id}                       parked command, steps, decisions
  audit/{id}                           append only
  rollups/{metric_yyyymmdd}            dashboard aggregates
  jobs/{id}                            import / generation / backfill progress
  outbox/{id}                          events for the worker
mcpTokens/{hash}                       tenantId, roleKey, label, revoked
```

6.2 Staying fast with dynamic fields

- **Stable field IDs**: renames are metadata-only, with no data rewrite.
- **Typed index slots**: filterable or sortable fields are assigned to `idx.s1-s8`, `n1-n6`, `d1-d6`, `r1-r4`. One fixed set of composite indexes serves every tenant and blueprint, so we never hit Firestore's index limit.
- **Computed fields materialised on write**, using the compiler's dependency graph. No read-time fan-out.
- **Dashboards read rollup documents**, never scan records. Count and sum aggregation queries cover ad-hoc questions.
- **Search**: prefix tokens for instant lookup; Firestore vector search with Gemini embeddings for fuzzy matching.

6.3 Isolation and correctness

Guarantee	How
Tenants never see each other	Path isolation under `tenants/{tid}`; Security Rules on custom claims; clients read only, never write; server builds every path through one tenant helper
Debits equal credits	Ledger engine is the only writer; refuses unbalanced entries; lines embedded so an entry is atomic
Postings immutable	Rules deny update and delete; corrections are reversing entries
Gap-free numbering	Counter incremented in the same transaction that issues the document

Guarantee	How
Stock matches moves	Move and level written in one transaction; nightly recompute and alert on drift
Exact amounts	Money in integer minor units; quantities in integer milli-units
No lost updates	Version field checked in every transaction

7 Approval engine

Approval flows are blueprint data, so any action can have one, defined by the owner in plain words and applied through the change studio.

```
approvals:
  - on: purchase_order.submit
    when: total > 500000
    steps:
      - role: production_manager
      - role: owner                       # sequential
  - on: stock.adjust
    when: abs(quantity) > 20 or channel == "mcp"
    steps: [ { any_of: [store_lead, owner] } ]
  - on: "*.delete"
    steps: [ { role: owner } ]
```

Feature	Behaviour	Depth
Triggers	Any command, record transition or engine action; wildcards	Prototype
Conditions	Safe expressions on amounts, quantities, parties, actor, channel	Prototype
Steps	Single, sequential, any-of a role or list	Prototype
Re-check on approve	The parked command is re-validated against current data before it runs	Prototype
Safety floors	Engine-defined minimums (refunds, posted-document voids, large stock write-offs)	Prototype
Agent polling	`get_approval_status(id)` over MCP	Prototype
Delegation, expiry, escalation	Approver on leave, time-outs, reminders	Roadmap

8 AI architecture

8.1 Agents

Agent	Job	Models
Architect: interview	Fixed checklist: products, production style, stages, materials, sales and purchasing flow, people, pain points. Uses files first; at most 8 questions; ends with a summary the owner confirms.	Gemini Flash
Architect: files	Spreadsheets: header + 50 rows per sheet, column typing, entity and relation inference, implied rules. Photos and PDFs of job cards, QC sheets, invoices: extract the fields they capture. Produces an evidence list.	Flash (vision), Gemma 4 for column typing
Architect: blueprint	Picks the industry pack, generates customisation overlays with structured output, validator repair loop (3 rounds), then a self-check against the evidence.	Gemini Pro
Change planner	Turns requests into overlay diffs with a plain-language preview; one clarifying question when ambiguous; refuses engine-breaking changes with a reason.	Flash, Pro for multi-entity
Import mapper	Column-to-field mapping with confidence, value cleaning, record linking by fuzzy match.	Flash, Gemma 4, Embedding
Operator assistant	Answers and acts through the generated tools only; numbers always from the database; shows which tools it used.	Gemini Flash (ADK)
Insights	Production plan from open orders and stock; reorder suggestions from BOM needs and lead times; scrap and defect patterns by machine, shift, operator. Outputs actions that can be applied (draft PO, draft job).	Gemini Pro + rollup data
Demo MCP agent	An ADK agent with `McpToolset` connected to the live factory endpoint.	Gemini Flash

8.2 Model routing

Model IDs live in one config file, per task, with a fallback each. Confirm availability on Vertex AI on day 1.

Task	Primary	Fallback
Blueprint design, complex changes, insights	Gemini Pro	Gemini Flash with more repair rounds
Interview, files, changes, assistant, MCP agent	Gemini Flash	Gemini Pro
Column typing, value cleaning	Gemma 4 on Cloud Run GPU	Gemini Flash-Lite
Intent routing, short labels	Gemini Flash-Lite	Flash
Embeddings	Gemini Embedding	—

Task	Primary	Fallback
Local-language labels (optional)	TranslateGemma	Gemini Flash

8.3 Guardrails (enforced in code)

- AI writes only blueprint overlays and tool calls. Never SQL, code or accounting logic.
- Every AI output is schema-validated before use.
- File contents, record values and tool results are wrapped as data; agent instructions forbid following text inside them.
- Risky actions park for approval regardless of who asks.
- Every AI decision that changes the system is logged with its inputs and the human approval.

9 MCP design

Every factory gets its own MCP server, generated from its live blueprint and regenerated on every change. The built-in assistant uses the same tool manifest: one definition, two surfaces.

Group	Generated from	Examples
Context	Blueprint	`describe_business` (also an MCP resource)
Records	Each entity	`search_production_job`, `get_customer`, `create_sales_order`, `update_supplier`
Workflow	Each transition and stage	`job_start_moulding`, `job_complete_stage`, `job_record_qc`
Engines	Core bindings	`receive_goods`, `issue_materials`, `receive_output`, `post_invoice`, `record_payment`, `adjust_stock`
Insights	Insights agent	`suggest_production_plan`, `suggest_reorders`, `get_metric`
System change	Change planner	`propose_change("add mould number to jobs")`, which always needs owner approval
Approvals	Approval engine	`get_approval_status`

Security

- Each token belongs to one tenant and one role; only tools the role can use are listed.
- Risky tools return `pending_approval` with an ID; the owner sees it in the approval queue.
- Rate limits per token; every call audited with the client name; tokens revocable in settings.
- A read-only demo token for judges.

10 Module catalogue

Every module and submodule, with its depth in the prototype. `P` = prototype, `R` = roadmap.

M0 · Platform and delivery

Submodule	Responsibility	Depth
Monorepo and tooling	pnpm workspaces, Turborepo, TypeScript strict, ESLint, Vitest	P
Firebase project	Auth, Firestore, Storage, App Hosting, Functions, Blaze plan, region	P
Google Cloud	Vertex AI, Cloud Run, Cloud Tasks, Secret Manager, GPU quota request, service accounts	P
Local environment	Firebase Emulator Suite, seed scripts, `.env` templates	P
Deploy pipeline	One command per service; staging and production	P
CI	Typecheck, unit tests, rules tests on every push	R
Observability	Cloud Logging, Trace, Error Reporting, AI token metering per tenant	P

M1 · Data layer `packages/db`

Submodule	Responsibility	Depth
Paths and converters	Typed path builders, Firestore converters per collection	P
Tenant transaction helper	`withTenant(tid, tx ⇒ …)`, retries, version checks	P
Security Rules	Read by membership and role scope; deny all client writes; Storage rules	P
Index catalogue	Fixed composite indexes over typed slots; vector index	P
Rules tests	Emulator tests for cross-tenant and write denial	P

M2 · Blueprint system `packages/blueprint`

Submodule	Responsibility	Depth
Schema	Zod schema for every blueprint part; JSON Schema export for Gemini structured output	P
Field-type catalogue	text, longtext, integer, decimal, money, quantity+unit, date, datetime, boolean, select, multiselect, ref, lines, file, image, phone, email, computed	P
Binding catalogue	Engine bindings: sales.*, purchasing.*, inventory.item, production.job, production.bom, ledger.*	P
Expression language	Tokenizer, parser, type-checker, evaluator; functions: count, sum, min, max, if, date maths, abs, round	P

Submodule	Responsibility	Depth
Layers and merge	Overlay operations, merge base + pack + overlays, conflict detection	P
Validator	All rules in §4.4, errors with JSON path and fix hint	P
Compiler	Slot assignment, dependency graph, permission matrix, record validators, approval matchers, tool manifest input	P
Diff	Version-to-version diff in plain operations, for previews and audit	P
Migration planner	Classify metadata-only vs data changes, reversibility, impact counts, backfill plan	P

M3 · Base blueprint and industry packs `packages/blueprints`

Submodule	Responsibility	Depth
Base manufacturing blueprint	Customers, suppliers, items, sales orders, deliveries, invoices, POs, GRNs, bills, payments, production jobs, BOMs, roles, dashboards	P
Pack: plastics	Demo depth, modelled on DP Plastics	P
Packs: garments, food, furniture	Enough to generate and show side by side	P
Chart of accounts templates	Manufacturing COA: raw materials, WIP, finished goods, GRNI, COGS, scrap loss, VAT	P
Pack marketplace	Partners publish packs	R

M4 · Core engines `packages/engines`

M4.1 Items and units

Items	Raw material, component, finished good, co-product, by-product, service; reorder level; lead time	P
Units	Any unit, per-item conversions (buy in rolls, use in metres, sell in dozens)	P
Variants	Size and colour matrices for garments	R

M4.2 Inventory

Locations and stages	Stores, WIP, finished goods, quarantine	P
Stock moves	Receive, issue, transfer, adjust, produce, scrap; append only	P
Levels and reservations	On hand, reserved for orders, available	P
Lots and expiry	Switched on per item by the blueprint	P
Valuation	Weighted average cost	P
FIFO, landed cost, cycle counts		R

M4.3 BOM and production

BOM	Single level; inputs, main output, co-products, by-products, expected scrap and yield; versioned	P
Production job	From a sales order (MTO) or a plan (MTS); planned quantity; BOM version; stages from the blueprint	P
Material issue	Raw → WIP, actual vs standard	P
Stages and QC	Stage transitions with QC checks from the blueprint; failed QC blocks or flags	P
Output and scrap	WIP → finished goods and co-products; scrap recorded with reason, machine, shift	P
Close and costing	Material cost rolled into outputs; usage variance	P
Multi-level BOM, routings with times, labour and overhead, capacity		R

M4.4 Sales

Customers	Credit limit, payment terms, price list	P
Sales order	Lines, reservation or MTO job creation, credit check	P
Delivery	Stock out, COGS posting	P
Invoice	Totals, tax lines, gap-free number, PDF, AR posting	P
Receipt	Payment against invoices, AR settlement	P
Quotations, partial deliveries, returns, credit notes		R

M4.5 Purchasing

Suppliers	Items supplied, lead time, terms	P
Purchase order	Lines, approval policy hook	P
Goods received note	Stock in, optional receipt QC, GRNI posting	P
Supplier bill	Match to GRN, AP posting	P
Payment	AP settlement	P
Purchase requests, three-way match tolerances, debit notes		R

M4.6 Accounting

Ledger	Balanced journal entries, append only, reversals	P
Posting rules	Fixed rules per engine event (table below)	P

Chart of accounts	From template, editable names and codes	P
Manual journals	Adjustments, with approval	P
Receivables and payables	Open items, ageing	P
Reports	P&L, balance sheet, trial balance, stock valuation	P
Tax	VAT rates as rules with effective dates	P
Bank reconciliation, periods and locking, multi-currency, tax filing packs		R

Event	Debit	Credit
Goods received	Raw materials inventory	Goods received not invoiced
Supplier bill	GRNI, VAT input	Accounts payable
Supplier payment	Accounts payable	Bank / cash
Material issue	Work in progress	Raw materials inventory
Output received	Finished goods (+ co-products)	Work in progress
Scrap	Scrap loss	Work in progress
Delivery	Cost of goods sold	Finished goods inventory
Sales invoice	Accounts receivable	Sales, VAT output
Customer receipt	Bank / cash	Accounts receivable

M4.7 Numbering, documents, permissions, approvals, audit

Numbering	Gap-free per document type; formats from the blueprint	P
Documents	Totals, tax lines; PDF for invoice, PO, job card via `@react-pdf/renderer`	P
Permissions	Role × action × row scope from the compiled matrix	P
Approvals	As §7	P
Audit	Who or which agent, what, when, channel, before and after	P

M5 · Runtime `packages/runtime`

Submodule	Responsibility	Depth
Command pipeline	As §5	P
Record service	Validate, store by field ID, fill slots, links, search tokens	P
Computed fields	Dependency-graph updates in the same transaction	P
Rules engine	Evaluate blueprint rules on create, update, transition	P

Submodule	Responsibility	Depth
Query planner	Filters and sorts → slot queries, pagination, aggregations	P
Rollups	Daily metric documents updated in the command transaction	P
Blueprint cache	Compiled blueprint by tenant + version, in-process	P
Outbox	Events written in-transaction, relayed by the worker	P

M6 · Tools `packages/tools`

Manifest generator	Blueprint → tools with JSON Schema inputs and descriptions (§9)	P
Role filter	Hide tools a role can't use	P
Executor	Tool call → command → result or pending_approval	P

M7 · AI platform `packages/ai`

Gen AI client	`@google/genai` on Vertex AI; AI Studio key for local	P
Model router	Task → model + fallback, from config	P
Structured output + repair	Schema-constrained calls; validator errors fed back	P
Prompt library	Versioned prompts per task, with examples	P
Data wrapping	Prompt-injection defence for files and tool results	P
Gemma client	Calls the Gemma Cloud Run service, falls back to Flash-Lite	P
Metering	Tokens and cost per tenant and task	P

M8 · Agents `apps/agents`

Interview agent	Checklist interview, summary	P
File understanding	Sheets, photos, PDFs → evidence list	P
Blueprint generator	Pack choice, overlays, repair, self-check	P
Change planner	Diff, preview, clarifying question, refusal	P
Operator assistant	Tools-only answers and actions	P
Insights agent	Production plan, reorders, scrap and defect analysis	P
Demo MCP agent	ADK + McpToolset against the live endpoint	P
Voice (Gemini Live), daily brief		R

M9 · Import

Readers	XLSX and CSV (SheetJS), Google Sheets (Picker + Sheets API), images and PDFs (Gemini)	P
Profiling	Column statistics and samples	P
Mapping proposal	Column → field with confidence; owner confirms	P
Cleaning	Dates, phones, currency, units (Gemma 4 batch)	P
Linking	Orders to customers, lines to items, by fuzzy match	P
Loader	Cloud Tasks job through the command pipeline, BulkWriter, progress doc	P
Import report	Imported, needs attention, one-click fixes	P
Opening balances	Stock and AR/AP opening entries	P

M10 · MCP server `apps/mcp`

Transport	Streamable HTTP per tenant	P
Token auth	Hash lookup, tenant + role, revocation	P
Tool listing and calls	From the manifest, through the executor	P
Resources	`describe_business`	P
Rate limits and audit	Per token	P
OAuth, per-tool scopes UI		R

M11 · Worker `apps/worker`

Job runner	Cloud Tasks handlers with idempotency and progress	P
Outbox relay	Firestore trigger → embeddings, tool regeneration, notifications	P
Backfills	Data-touching blueprint changes	P
PDF rendering	Documents to Cloud Storage	P
Integrity checks	Nightly ledger balance and stock recompute (Cloud Scheduler)	P

M12 · Design system `packages/ui`

Tokens	Neutral palette, one accent, light and dark, spacing, radius, Inter via Google Fonts	P
Primitives	Radix + Tailwind v4: button, input, select, dialog, sheet, tabs, toast, command palette	P
Data table	TanStack Table: dense, keyboard, filters, saved views, bulk actions	P
Form renderer	Any blueprint entity → form, including line grids and computed totals	P
Views	Detail, kanban by stage, calendar, dashboard widgets, charts	P

AI components	Chat, tool-call trace, diff and impact viewer, evidence list, confidence chips	P
States	Empty, loading skeletons, error, permission denied	P

M13 · Web app screens `apps/web`

Screen	Contents	Depth
Landing	One sentence, "Open the demo factory", "Build mine"	P
Sign in	Firebase Auth (Google, email link); demo mode without sign-in	P
Onboarding	Chat interview, file drop, live "what I've learned" panel	P
Blueprint review	Modules, records and links, stages as flows, approvals, roles; approve or ask for changes	P
Import	Mapping with confidence, sample preview, import report	P
Home dashboard	What happened, what needs me, what's next	P
Module workspaces	Sales, purchasing, inventory, production (kanban by stage), accounting; list, detail, form views	P
Production floor view	Jobs by stage and machine, QC entry, scrap entry	P
Insights	Production plan, reorder suggestions, scrap analysis; apply as drafts	P
Reports	P&L, balance sheet, trial balance, AR/AP ageing, stock valuation	P
Change studio	Ask, preview with before and after and data impact, apply, version history, undo	P
Assistant panel	Available everywhere; shows tools used	P
Approvals	Queue from staff and agents; approve or reject with context	P
Agents and MCP	Endpoint URL, tokens per role, recent agent activity	P
Settings	Members and roles, company, currency, documents, language	P
Audit log	Filter by actor, agent, channel, record	P

M14 · Proof and submission

Eval suite	Datasets and runner for §13 targets	P
Demo factory data	3 months of plastics orders, POs, jobs, QC, scrap	P
Deck (PDF)	Problem, solution, impact, feasibility, scalability, architecture, models, eval results, roadmap, team	P
Video (3 min)	Recorded in segments	P
README	Setup, architecture, Google services used	P

11 Build order

Contracts first, so three people can work in parallel without waiting on each other. Then the critical path, which runs through the blueprint, runtime and generic UI before anything AI-generated can be seen working.

11.1 Contracts frozen first (first 3 hours)

Contract	Used by
Blueprint schema types (Zod)	Everything: AI output, runtime, UI, tools
Command and result types	Web, assistant, MCP, worker
Tool manifest type	MCP, assistant, demo agent
Firestore layout and converters	Engines, runtime, web listeners
Design tokens	Every screen

11.2 Critical path

Schema → Validator + compiler → Base blueprint → Command pipeline →
Generic list / form / detail → Blueprint generation → Import → Change planner → Tools + MCP

11.3 Ordered steps

#	Step	Depends on	Lane
1	Contracts (11.1)	—	All
2	Monorepo, Firebase and GCP setup, deploy skeletons, GPU quota request	—	B
3	Design system tokens, primitives, app shell	1	C
4	Blueprint schema, catalogues, expression language	1	A
5	Data layer, Security Rules, index catalogue	1, 2	B
6	Layers and merge, validator, compiler	4	A
7	Base manufacturing blueprint + plastics pack	6	A
8	Command pipeline, record service, permissions, audit	5, 6	B

#	Step	Depends on	Lane
9	Data table, form renderer, detail, workspace sidebar	3, 7	C
10	AI platform: client, router, structured output + repair	2	A
11	Items, units, inventory engine, numbering	8	B
12	Interview + file understanding + blueprint generation	7, 10	A
13	Onboarding and blueprint review screens	9, 12	C
14	Import: readers, mapping, cleaning, loader, report	8, 10	A
15	Ledger and posting rules; sales and purchasing engines	11	B
16	BOM and production engine, QC, scrap	11, 15	B
17	Kanban by stage, calendar, dashboard, rollups	9, 8	C
18	Approval engine + approvals screen	8	B / C
19	Diff, migration planner, change planner, change studio, undo	6, 12	A / C
20	Tool generation, MCP server, tokens screen	8, 18	B
21	Operator assistant + assistant panel	20	A / C
22	Insights agent + insights screen	16, 17	A / C
23	Documents and PDFs; reports (P&L, ageing, stock value)	15	B / C
24	Gemma service for import cleaning	2, 14	A
25	Demo MCP agent (ADK + McpToolset)	20	A
26	Demo data, garments / food / furniture packs	7, 16	C
27	Eval suite run and fixes	all	All
28	Deck, video, README, submission	all	All

Lanes: A = AI and agents · B = backend and engines · C = frontend and story.

12 Three-day schedule

Times are Sri Lanka time. Submit by the afternoon of day 3 with buffer. Each day ends with the deployed version passing its exit check.

Day 1 · Foundations and the first generated factory

Time	A: AI and agents	B: Backend and engines	C: Frontend and story
08:00–11:00	Contracts; blueprint schema; confirm model access on Vertex AI	Contracts; monorepo; Firebase + GCP; deploy skeletons; GPU quota	Contracts; tokens; primitives; app shell; sign in
11:00–15:00	Expression language; layers and merge; validator; compiler	Data layer; rules; command pipeline; record service; audit	Data table; form renderer; detail view; sidebar from blueprint
15:00–19:00	Base blueprint + plastics pack; AI client + repair loop	Permissions; items, units, inventory; numbering	Onboarding chat and file drop
19:00–22:00	Interview, files, blueprint generation; 3 test factories valid	Import loader job; deploy all	Blueprint review screen; smoke test

> **Exit:** describe a plastics factory, upload its spreadsheet, get a working system with records, deployed on App Hosting and Cloud Run.

Day 2 · Engines, change, agents

Time	A: AI and agents	B: Backend and engines	C: Frontend and story
08:00–12:00	Import mapping, cleaning, linking; change planner	Ledger + postings; sales and purchasing engines	Kanban by stage, dashboard, rollups; import screen
12:00–16:00	Diff + migration planner; insights agent	BOM + production, QC, scrap; approval engine	Change studio; approvals screen; production floor view
16:00–19:00	Operator assistant on shared tools	Tool generation; MCP server; tokens	Assistant panel; insights screen; agents screen
19:00–22:00	Demo MCP agent; Gemma service	Invoice and PO PDFs; reports	Demo factory data (3 months); other packs

> **Exit:** the full demo story works end to end once, on the deployed version.

Day 3 · Proof, polish, submit

Time	Everyone
08:00–10:00	Run the eval suite; fix the top failures
10:00–12:00	Polish the demo path; accessibility pass (keyboard, contrast); feature freeze at 12:00
12:00–15:00	Record the video in segments; write the deck and README
15:00–17:00	Final deploy; check judge access from a clean browser; submit
17:00 onward	Buffer

13 Evaluation and cut list

13.1 Evaluation targets

Area	Test	Target
Blueprint generation	16 factory descriptions across plastics, garments, food, furniture	15 of 16 valid within 3 repair rounds
Blueprint fit	Two reviewers score fit (1–5)	Average 4+
Import mapping	5 messy spreadsheets with known mapping	95%+ columns correct
Change requests	20 requests on the demo factory	18 of 20 correct, applicable diffs
Engine safety	Requests to alter postings or skip approvals	0 succeed
Ledger integrity	Trial balance after demo data and eval runs	Balances to zero
MCP	25 agent tasks	23 of 25 correct; all risky ones held
Latency	First blueprint; change preview; assistant answer	< 60 s; < 15 s; < 5 s

13.2 Cut list, in order, if behind

1. Calendar view
2. Gemma service (keep Flash-Lite; Gemma in the deck)
3. Garments, food and furniture packs beyond generation proof
4. Balance sheet (keep P&L and ageing)
5. PDF documents (show the record instead)
6. Assistant panel in the web app (keep the MCP agent)
7. Undo (keep preview and apply)

> **Never cut:** blueprint generation, import, the sales → production → purchasing → accounting loop, change with preview, dynamic approvals, MCP with approvals, insights agent.

14 Google AI Builder Cup compliance

14.1 Mandatory requirements

Requirement	Covered by
Google AI models (Gemini or Gemma)	Gemini Flash, Pro, Flash-Lite, Embedding on Vertex AI; Gemma 4 on Cloud Run
Agentic platforms	Agent Development Kit (TypeScript); AI Studio for prompt work
Deployed on Cloud Run or Firebase	Firebase App Hosting + Cloud Run services
Working prototype	Public demo factory, no sign-up for judges; public MCP demo token
Proposal deck as PDF	Day 3
Public 3-minute video	Day 3
Problem statement	**Manufacturing: Intelligent Operations & Industrial Efficiency**
English materials	All

14.2 Google stack

Hosting	Firebase App Hosting, Cloud Run (agents, mcp, worker, Gemma GPU)
Data	Cloud Firestore (with vector search), Cloud Storage for Firebase
Identity and security	Firebase Auth, App Check, Secret Manager, IAM
Background	Cloud Tasks, Cloud Functions 2nd gen, Cloud Scheduler
AI	Gemini on Vertex AI, Gemma 4, ADK, AI Studio
Workspace	Google Sheets API, Google Picker
Operations	Cloud Logging, Trace, Error Reporting, Monitoring

14.3 Judging criteria

Criterion	Weight	What judges see
Technical merit and Gen AI	40%	Layered blueprint with schema-constrained generation and repair; multimodal reading of job cards and QC sheets; safe migrations; ADK agents; generated MCP; Gemini + Gemma routing; measured eval results
Problem alignment and impact	25%	Small manufacturers on paper and Excel get production, stock, sales, purchasing and books in minutes; insights cut scrap and stock-outs
Innovation and creativity	25%	A factory system that shapes itself per industry; fixed-core engines; per-factory MCP with approvals; agents that can propose system changes

Criterion	Weight	What judges see
UX and solution design	10%	Plain-language setup and changes; previews before anything changes; one consistent design system; keyboard and screen-reader accessible; live updates

14.4 Demo script (3 minutes)

Time	On screen	Point
0:00–0:20	A plastics factory's reality: paper job cards, an Excel order sheet, a stock-out	The problem
0:20–1:00	Owner describes the factory, uploads the sheet and a photo of a job card; a working system appears with real data	Generate and import
1:00–1:30	Sales order → production job on the stage board → QC → deliver → invoice; P&L updates	A real system
1:30–1:55	"Add a regrind stage and POs over 500k need my approval." Preview, approve	Changes safely by conversation
1:55–2:35	Gemini agent over MCP: "What should we produce this week?" then "Order the resin we need." The PO is held for approval	Agent-ready, with guardrails
2:35–2:50	Garment unit and food processor generated side by side	Dynamic for any manufacturer
2:50–3:00	Architecture: fixed core, AI-shaped blueprint, Gemini, Gemma and ADK on Google Cloud	Depth and vision

15 Risks and open decisions

15.1 Risks

Risk	Impact	Mitigation
Scope for 3 days	High	Contracts first; prototype depths fixed in §10; cut list; feature freeze at noon on day 3
Generated blueprints wrong	High	Base + packs mean AI only writes customisations; strict validator; repair loop; owner review
Ledger or stock errors without SQL constraints	High	Single-writer engines in transactions; rules deny client writes; nightly integrity checks; ledger tests
Data loss during changes	High	Stable field IDs; metadata-only changes where possible; backfills keep old values; undo
Agent misuse	High	Role-scoped tokens; approval engine; safety floors; rate limits; audit
GPU quota denied	Medium	Gemma via Model Garden, or Flash-Lite with Gemma in the deck
Model IDs change	Medium	IDs in config with fallbacks; confirm on day 1
Prompt injection via files	Medium	Data wrapping; schema validation; AI can't act outside tools
Seen as "an app builder"	Medium	Lead with fixed-core engines, safe evolution and agent-ready factories

15.2 Open decisions

1. Gemini model IDs available on the Vertex AI project.
2. Firebase / GCP project ID, region and Blaze plan.
3. Team size and lane owners.
4. Permission to use DP Plastics as the model for anonymised demo data.
5. Demo local language (Sinhala or Tamil), if any.